

Authentication & User Management

FRD §17 — Administrator manages users. Version 1.0 · Base URL `http://<host>:7100`

1. Conventions

All requests use JSON. Every endpoint except `POST /api/auth/login` requires a bearer token:

```
Authorization: Bearer <jwt>
```

Response envelope

Shape	Body
Success	<code>{ "success": true, "message": "...", "data": { ... } }</code>
Paginated	<code>{ "success": true, "data": [...], "meta": { page, pageSize, total, totalPages } }</code>
Error	<code>{ "success": false, "error": "message" }</code>

Status	Meaning
200 / 201	OK / Created
400	Validation error (see <code>error</code>)
401	Missing / invalid / expired token, or wrong current password
403	Authenticated but not an Administrator
404	User not found
409	Email already in use

Tip: the `password_hash` is **never** returned by any endpoint. Numeric fields elsewhere in the API come back as JSON strings, but user fields here are plain types.

2. Roles

Six roles (the `role` field). **Administrator** bypasses all role checks.

Role	Responsibility
Administrator	Manage masters, users & configuration — full access
Engineering	Create materials & BOMs, submit for review
Commercial	Review pricing, commercial approval, quotation comparison
Purchase	Vendors, RFQs, purchase orders
Stores	Goods receipt, inventory, stock transfers
Management	Dashboards, reports, analytics

3. Authentication

POST `/api/auth/login`

public

Validates credentials and returns a JWT (HS256, default 12h expiry).

Request

```
{ "email": "admin@innopolis.bio", "password": "admin123" }
```

Response 200

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": { "id": "uuid", "name": "Priya Nair", "email": "admin@innopolis.bio",
              "role": "Administrator", "team": "Admin", "initials": "PN", "hue": 280 }
  }
}
```

Store the token; send it on every subsequent request. On `401`, redirect to login.

GET /api/auth/me

any authenticated

Returns the current user (same shape as `data.user` above).

POST /api/auth/set-password

authenticated · flagged only

Forced first-login change. The user supplies the temporary password they were given plus their new password. This is the *only* way a user sets their own password — there is no routine self-service change.

Request

```
{ "current_password": "<the temporary password>", "new_password": "their-own-secret" }
```

Response 200

```
{
  "success": true, "message": "Password updated",
  "data": {
    "token": "eyJ...", // fresh, UNRESTRICTED token – use this now
    "user": { "id": "uuid", "must_change_password": false, ... }
  }
}
```

Wrong temporary password → `401`. Called when the account is **not** flagged → `403` ("Password changes are managed by your administrator").

The `must_change_password` flag & the gate. Login returns `data.must_change_password`. When `true`, the token is **restricted**: every endpoint except `/api/auth/login`, `/api/auth/set-password`, `/api/auth/me`, `/health` returns HTTP `403` with `{ "code": "PASSWORD_CHANGE_REQUIRED" }`. **Frontend:** on login, if the flag is `true` (or on any 403 with that code), send the user to a "set new password" screen → call `set-password` → store the returned token and continue.

4. User Management (Administrator only)

All endpoints below require an **Administrator** token. A non-admin gets `403`.

Create a user. The user logs in with the `password` you supply — **no forced change on create**. (To require a change, use reset-password below.) `initials` are auto-derived from the name if omitted.

Request

```
{
  "name": "Sara Khan",
  "email": "sara@innopolis.bio",
  "password": "Sara-2026!",           // the user's initial password (no forced change)
  "role": "Purchase",                 // one of the 6 roles
  "team": "Procurement",             // optional
  "initials": "SK",                  // optional (auto from name)
  "hue": 210                          // optional avatar hue 0-360
}
```

Response 201

```
{
  "success": true, "message": "User created",
  "data": { "id": "uuid", "name": "Sara Khan", "email": "sara@innopolis.bio",
            "role": "Purchase", "team": "Procurement", "initials": "SK", "hue": 210,
            "is_active": true, "created_at": "...", "updated_at": "..." }
}
```

Required: `name`, `email`, `password`, `role`. Duplicate email → `409`.

GET /api/users

Administrator

Paginated list. Query params:

Param	Notes
search	matches name or email
role	filter by exact role
page / pageSize	default 1 / 50 (max 200)

```
GET /api/users?role=Purchase&search=sara&page=1&pageSize=50
```

Returns `data: [user]` + `meta: { page, pageSize, total, totalPages }`.

GET /api/users/:id

Administrator

Fetch a single user by id. `404` if not found.

PATCH /api/users/:id

Administrator

Update profile fields only. **Passwords are not set here** — use reset-password below.

Request (all optional)

```
{
  "name": "Sara A. Khan",
  "email": "sara.khan@innopolis.bio", // re-checked for uniqueness
  "role": "Stores",
  "team": "Stores",
  "hue": 152,
  "is_active": false // deactivate without deleting
}
```

Response 200 — the updated, sanitized user.

POST /api/users/:id/reset-password

Administrator

Reset a user to a **temporary password** and flag them to change it at next login. Supply your own, or omit `temporary_password` to have one generated and returned.

Request (body optional)

```
{ "temporary_password": "Temp-2026!" } // optional; omit to auto-generate
```

Response 200

```
{
  "success": true, "message": "Temporary password set",
  "data": {
    "user_id": "uuid",
    "temporary_password": "Tmp-9fK2aQ7p", // shown ONCE – share it with the user
    "message": "Temporary password set. Share it with the user; they must change it at next login."
  }
}
```

The reset replaces the old password — the user can no longer log in with it. They sign in with the temporary password, then are forced to `set-password`.

DELETE /api/users/:id

Administrator

Soft-delete (deactivate). The user is retained in the database but `is_active` becomes false and they can no longer log in.

```
{ "success": true, "message": "User deactivated", "data": { "message": "User deactivated successfully" }
```

5. Safety guards

These rules return `400` to prevent locking yourself or the organization out:

- You **cannot deactivate or delete your own account**.
- The **last active Administrator** cannot be demoted, deactivated, or deleted.

6. Field reference

Field	Type	Notes
<code>id</code>	string (uuid)	generated
<code>name</code>	string	1–120 chars
<code>email</code>	string	unique, lowercased, login identity
<code>password</code>	string	write-only, min 6 chars, bcrypt-hashed
<code>role</code>	enum	Administrator · Engineering · Commercial · Purchase · Stores · Management
<code>team</code>	string?	free text label
<code>initials</code>	string?	≤ 4 chars, auto-derived from name
<code>hue</code>	integer	0–360, avatar colour
<code>is_active</code>	boolean	false = cannot log in
<code>must_change_password</code>	boolean	true = restricted token until <code>set-password</code>
<code>created_at</code> / <code>updated_at</code>	string (ISO)	timestamps

7. Typical flow

1. Admin logs in POST `/api/auth/login` (admin@innopolis.bio)
2. Create the team POST `/api/users` (set each person's password)
→ user logs in directly with that password (no forced change)
3. Forgot / rotate password POST `/api/users/:id/reset-password` (admin issues a temporary password)
→ user logs in POST `/api/auth/login` (with temp → `must_change_password:true`)
→ forced change POST `/api/auth/set-password` (temp + new → fresh token)
4. Manage / audit GET `/api/users?role=...` PATCH `/api/users/:id`
5. Off-boarding DELETE `/api/users/:id` (deactivate)